Redundancy and Replication for Availability

copies only help if they fail separately

Watch the 4-min video

In one line: run at least N+1 (better N+2) copies, place them in different failure domains, usually three availability zones each able to carry the load without one zone, and replicate the data too, because copies that share power, a deploy or a dependency fail together.

1 Two copies, one building

You run two copies of every service and feel safe. Then one data centre loses power and both copies go down, because they were in the same building. Redundancy only adds availability when the copies fail **independently** (lesson 1's parallel formula assumes exactly that).

2 How many copies: N, N+1, N+2

N is what you need to handle peak load. **N+1** adds one spare, so you survive one failure. **N+2** survives a failure while another instance is down for a deploy, patching or maintenance, which happens every week. With only N+1, a routine deploy plus one crash already leaves you short.

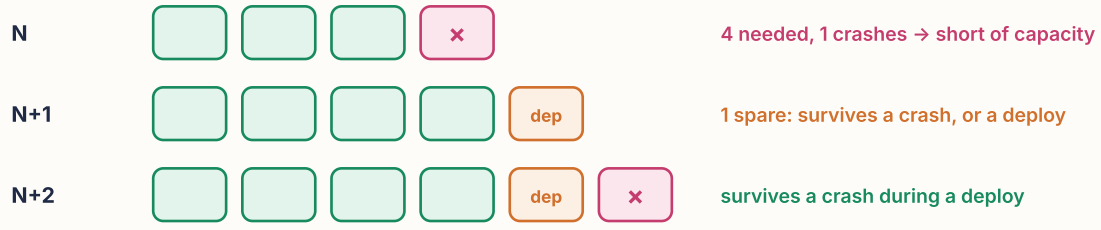


Figure 1. With N = 4: N+1 covers one event at a time; N+2 covers the common case of a crash during planned work.

3 Failure domains

A **failure domain** is a set of things that fail together because they share something. Redundant copies must sit in **different** domains.

Failure domain	Shared thing	Spread by
Host / rack	power supply, top-of-rack switch	anti-affinity, spread placement groups
Availability zone	building(s), power, cooling, network	instances in ≥ 3 zones
Region	a metro area, regional control plane	multi-region (lesson 15)
Software version	one bad deploy hits every copy	staged rollouts, canaries (lesson 75)
Config / dependency	one config push, one DNS provider, one DB	staged config, diverse or redundant dependencies

In the cloud, an **availability zone** is one or more data centres with independent power, cooling and networking, a few milliseconds from the others in its region. Use **three zones** and size each so the other two carry the full load: each zone needs 50% of peak, so you run 150% of N in total.

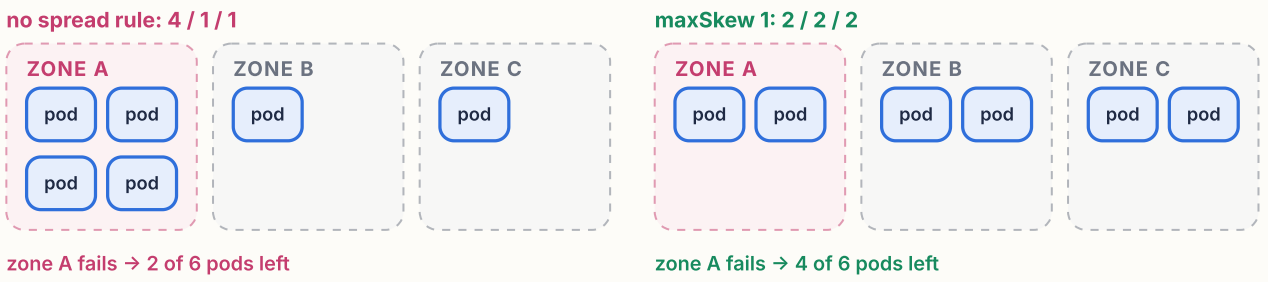


Figure 2. The same 6 pods. Without a spread rule the scheduler can pack a zone; losing it takes most of your capacity.

4 Correlated failures, and the data

The hidden danger is **correlation**: a bad deploy pushed everywhere at once, a shared dependency, even disks from the same batch wearing out together. Fight it with staged rollouts, diverse dependencies and independent copies of critical state.

Stateless services are easy to copy; **data needs replication**. Keep database replicas in other zones: **synchronous** to lose nothing when a zone fails, or **asynchronous** for lower write latency with a small loss window (lesson 13). Object stores such as S3 Standard already store data across multiple zones. Your availability is capped by the **least redundant** component on the request path.

WORKED EXAMPLE: SIX PODS ACROSS THREE ZONES

A web service runs 6 pods on Kubernetes. Without guidance the scheduler might put 4 in zone A. A **topology spread constraint** with `topologyKey: topology.kubernetes.io/zone` and `maxSkew: 1` forces 2 per zone. If zone A fails, 4 pods keep serving and the autoscaler adds more in B and C. A **PodDisruptionBudget** (e.g. `minAvailable: 5`) stops node maintenance from draining too many at once.

```
# Deployment pod spec: spread 6 replicas evenly over zones
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: { matchLabels: { app: web } }
```

Figure 3. The spread rule from the example. `DoNotSchedule` leaves a pod pending rather than break the spread.

COMMON MISTAKES

- Replicas in one zone, rack or node, counted as independent.
- Running exactly N+1, so every deploy is a single-failure-from-outage window.
- Redundant app servers in front of a single database instance.
- Zones not sized to absorb a lost zone, so the survivors overload and fail too.

5 Key terms

N+1 / N+2: capacity for peak plus one or two spares

Availability zone: isolated data centre(s) within a region

Topology spread: Kubernetes rule to balance pods across zones

Failure domain: components that fail together

Correlated failure: one cause takes down several copies

PodDisruptionBudget: limit on pods taken down voluntarily at once

6 Interview questions

Q1. What is a failure domain? BASIC

A set of components that can fail together because they share something: power, a switch, a rack, a zone, a region, a software version or a config push. Redundant copies must sit in different ones.

Q2. How much redundancy do you need? BASIC

At least N+1, and N+2 to survive a failure during maintenance or a deploy, spread across at least three zones with each zone sized so the other two can carry the load.

Q3. You need 6 instances at peak across 3 zones. How many do you run? SCENARIO

Each zone must carry the load if one fails, so 2 surviving zones need 6: 3 per zone, 9 in total.

Q4. Your app servers are in three zones. What else could still take you down? SCENARIO

Anything shared: a single-zone database, one DNS or identity provider, a global config push or a deploy that reaches every zone at once.

Q5. Why is N+1 often not enough? TRICKY

Planned work regularly takes one instance out; a single crash at that moment leaves you below N, so N+2 covers the realistic case.

Q6. Does replicating data synchronously across zones remove all data-loss risk? TRICKY

No: it protects against losing a zone, but a bad write, bug or deletion is replicated instantly too; you still need backups (lesson 14).

Next: SD-K-011 · Failover Mechanisms backendroadmap.com